"ROBUST AUDIO WATERMARKING FOR SECURE AUDIO AUTHENTICATION"

TEAM C1
ISHWARYA - 24220
HIMA VARSHINI - 24228
MEGHANA K - 24232
RANJITH RAJA B - 24250

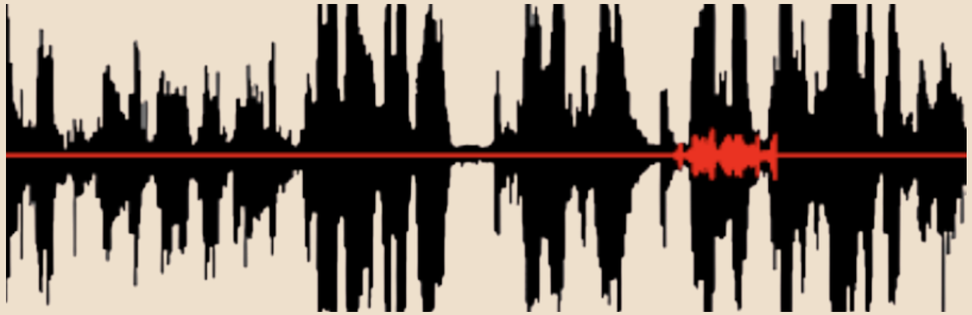
PROBLEM STATEMENT

"The problem addressed in this work is the design of a robust and imperceptible audio watermarking system capable of embedding authentication information within an audio signal such that the embedded watermark remains undetectable to human listeners while being resilient to common signal processing attacks such as noise addition, compression, cropping, and filtering."

INTRODUCTION TO WATERMARKING

A digital watermark is hidden information embedded inside a host signal (audio/image/video) for:

- Copyright protection
- Ownership verification
- Tamper detection
- Authentication

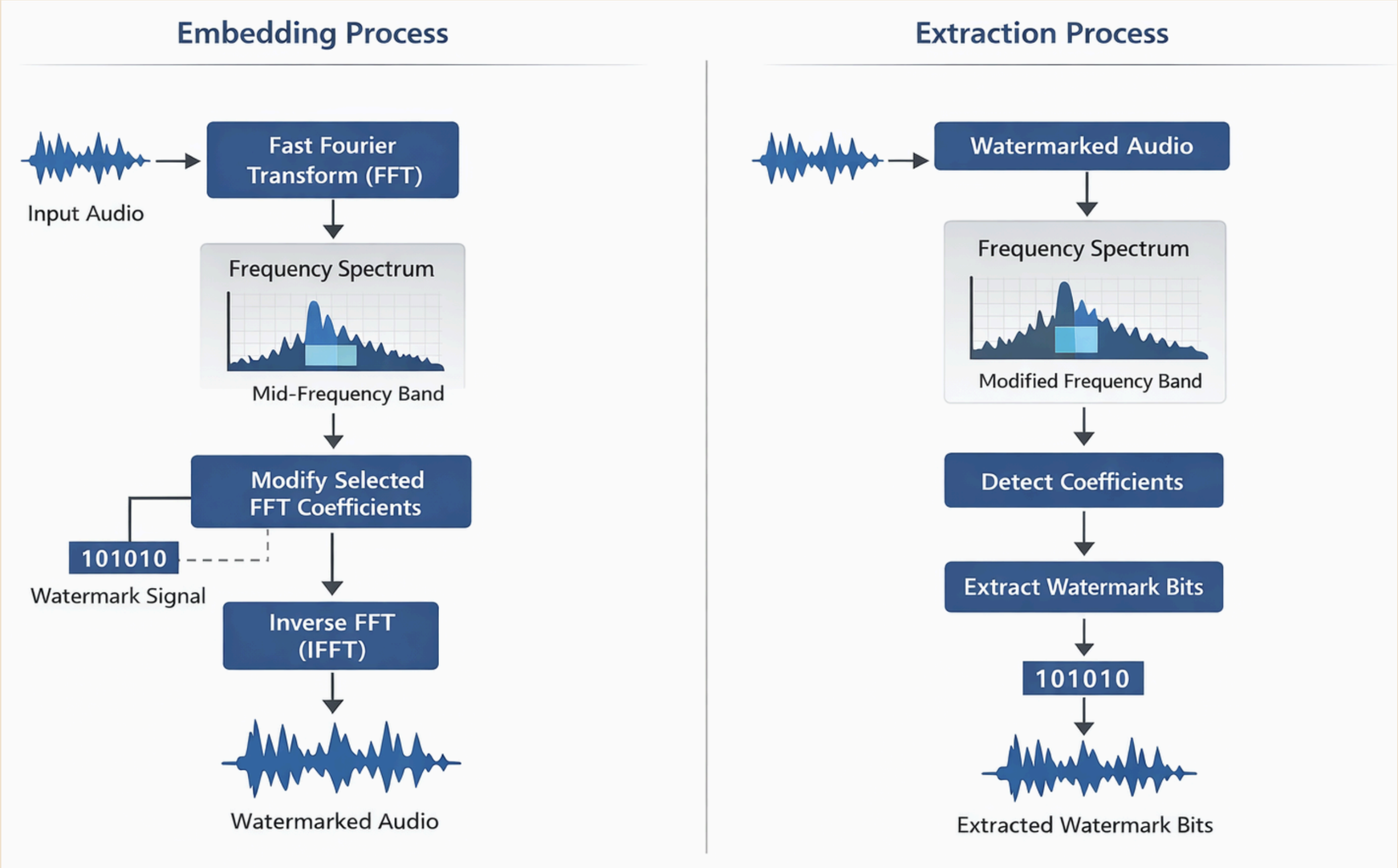


In Audio:

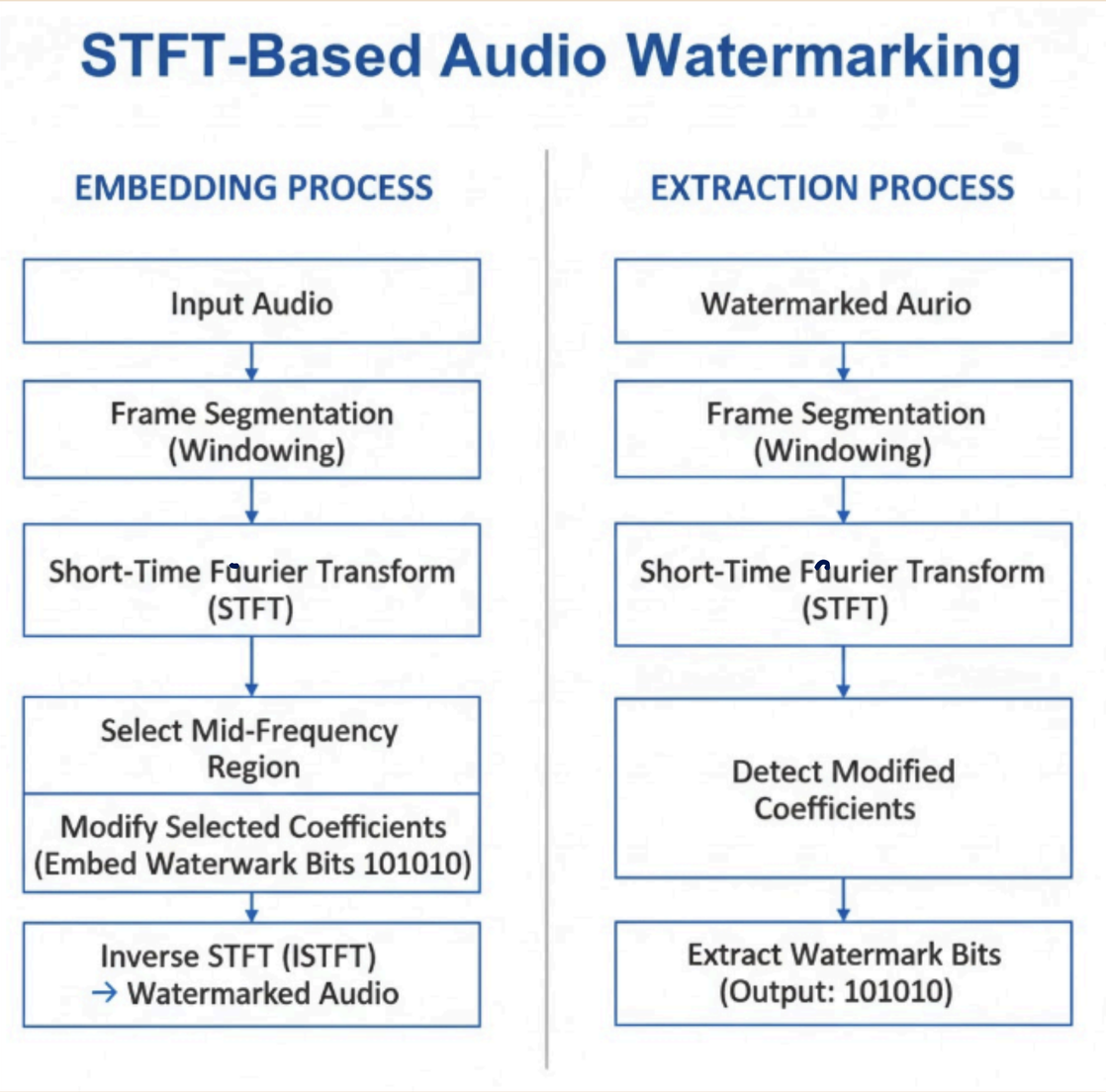
Audio watermarking hides data inside an audio signal such that:

- It is imperceptible to human hearing
- It is robust to attacks
- It can be extracted or verified later

INITIAL METHOD 1: FFT-BASED WATERMARKING



INITIAL METHOD 2: STFT-BASED WATERMARKING



WHY A NEW ARCHITECTURE WAS NECESSARY ???

Limitation of Existing Approaches

FFT/STFT-based watermarking focuses mainly on:

- Selecting frequency bands
- Adjusting embedding strength
- Improving robustness against noise

However, these methods:

- Do not cryptographically bind identity
- Are vulnerable to AI-based regeneration
- Lack structured recovery under severe distortion

THE AI "ARMS RACE" (CURRENT TRENDS/THREATS)

1. **Embedding in multiple domains**
2. **Encoder decoder nets**
3. **Generative/ Adversarial models**
4. **Attention based Architecture**

PHASE 1 – STANDARDIZATION & SIGNAL PREPARATION

Audio inputs vary in:

- Sampling rate (8 kHz – 96 kHz)
- Bit depth (8 / 16 / 24 bit)
- Stereo vs Mono
- Lossy vs Lossless formats

These variations affect:

- FFT coefficient stability
- Embedding strength
- Extraction reliability

Parameter	Standardized Value
Sampling Rate	44.1 kHz
Channels	Mono
Bit Depth	16-bit PCM
Normalization	Amplitude normalized

PHASE 2 – AUDIO HASHING (CONTENT FINGERPRINT)

1. Signal Representation:

   Let the standardized audio be:

$$x[n], \quad n = 0, 1, 2, \ldots, N-1$$

2. Hash Computation

$$H = \text{SHA-256}(x[n] \,||\, ID \,||\, T)$$

Where:

- ID = Owner Identifier
- T = Timestamp
- || = Concatenation

3. Even if the same audio is used:

- Different OwnerID → Different Hash
- Different Timestamp → Different Hash

4. Integrity Verification

Compute

$$H' = \text{SHA-256}(x'[n] \,||\, ID \,||\, T)$$

Check:

$$H' \stackrel{?}{=} H$$

- If equal → Authentic
- If not equal → Tampered

PHASE 3 – DIGITAL SIGNATURE USING PRIVATE KEY

Step 1 – Key Pair Generation

- A random private key (sk) is generated.
- A corresponding public key (pk) is derived mathematically from it.

Private key → kept secret

Public key → shared for verification

Step 2 – Signature Generation

Given message (hash) H:

1. A temporary random value is generated.
2. It is combined with:
   - The private key
   - The message H
3. Elliptic curve operations are performed.
4. A signature S is produced.

$$S = \mathrm{Sign}_{sk}(H)$$

Signature size ≈ 64 bytes.

PHASE 3 – DIGITAL SIGNATURE USING PRIVATE KEY

Step 3 - Public Verification

Anyone with the corresponding public key (pk) can verify:

$$\mathrm{Verify}_{pk}(H, S) = \mathrm{True}$$

If true:

- Audio unchanged
- Metadata authentic
- Ownership confirmed

Embedded Payload: The watermark carries the signed fingerprint, not raw metadata.

$$P = (H, S)$$

PHASE 3 – WORKING OF ED25519

Key Generation

Private Key (sk) ⟶ 32-Byte Secret

Public Key: $pk = aG$

Signature Generation

Public Key (pk)
Hash H,
Signature (R, s)

Nonce Generation
$r = Hash(sk, H)$

Compute Point
$R = rG$

Calculate Challenge
$k = Hash(R, pk, H)$

Compute Signature
$s = r + k\,a$
$S = (R, s)$

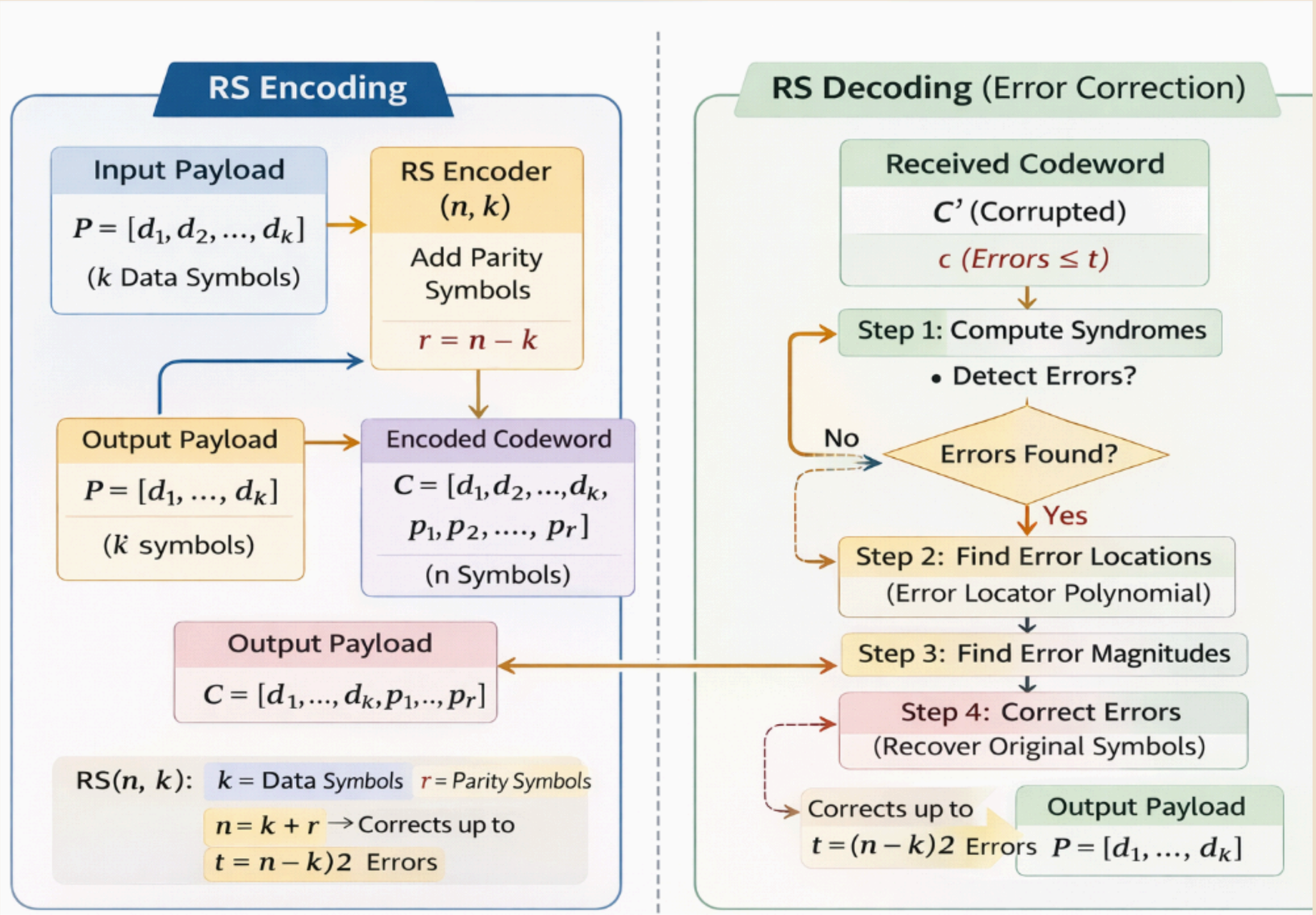
Verification

Public Key (pk)
Hash H,
Signature (R, s)

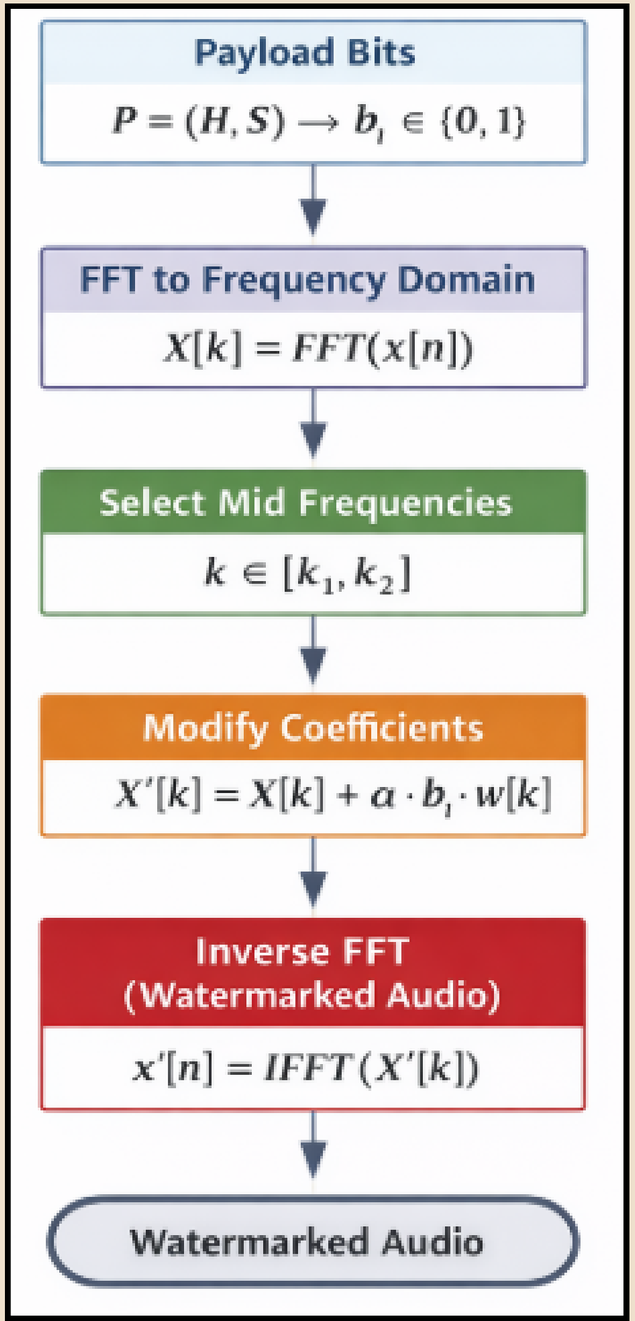
Verify: $sG = R + k(pk)$

- *Signature Valid*
- *Message Intact*
- *Authorship Verified*

PHASE 4 - REED SOLOMON (ERROR CORRECTION)



PHASE 5 - EMBEDDING THE WATERMARK



PHASE 6 — WATERMARK EXTRACTION & AUTHENTICATION

1. **Receive Audio**
   Possibly attacked / modified signal.
2. **Standardize Format**
   Convert to required sampling rate and mono format.
3. **Apply FFT**
   Transform audio to frequency domain.
4. **Select Mid-Frequency Band**
   Isolate the same band used for embedding.
5. **Detect Embedded Pattern**
   Use correlation to recover hidden bits.
6. **Reconstruct Bitstream**
   Combine detected bits into encoded payload.
7. **Reed–Solomon Decode**
   Correct any corrupted symbols.
8. **Verify Signature**
   Validate authenticity using public key.

PHASE 7 – ATTACKS

Compression

- 320 kbps
- 128 kbps
- 64 kbps

Test lossy compression

Noise Addition

- White noise
- Background noise

Add background interference

Filtering

- Low-pass filter
- High-pass filter

Remove frequency bands

Cropping/ Trimming

- Cut audio segment
- Simulate loss

Simulate partial signal loss

Re-recording

- Playback via speaker
- Record via mic

Playback and capture attack

Test watermark robustness against common signal attacks

- BER
- SNR
- Detection Accuracy

THANK YOU